

Mini Project DSAI 427

Mohamed Hozien	202201507
Ziad Moutaz	202201252

March 18, 2026

1 Introduction

1.1 Inverted Index

An Inverted Index (also known as a Reverse Index) is a type of data structure in information retrieval systems that is used to index the words (terms) in one way, and the documents containing those words in the other. The system maintains a list of all the documents containing the given word instead of having to store the documents and then go through them searching the word.

This system is of great value to different users, which include:

- **Librarians:** To handle and index large digital archives and book collections.
- **Researchers:** To find certain terms, themes or citations in large volumes of literature in the public domain.
- **Search Engine Users:** To get close real-time search results on huge corpus of web or documents.

1.2 Sequential scanning and its limitations

Sequential scanning involves searching all the documents in a collection of documents to determine instances of a particular word or query. This method is ineffective since it treats documents individually and hence time complexity is high particularly when the dataset increases. The search time is linear with the number of documents, and thus not feasible in large-scale data. Also, sequential scanning fails to make use of parallelism or indexing structure, which makes query responses slower than more optimized methods such as inverted indexing.

2 Data Science Lifecycle Implementation

2.1 Storage Analysis

Each book that is contained in HDFS is a file in the form of .txt. HDFS breaks down the file into blocks when uploaded (the default size is 128 MB in Hadoop 3). Because the books of Gutenberg are not that large, the majority of books must probably take up one block. HDFS enhances block replication as a method of enhancing fault tolerance and reliability. The factor of replication ensures that each block is copied to several DataNodes. Hadoop attempts to put computation in the node where the data is instead of transferring large data through the network.

2.2 Preparation Impact

These are highly used words that occur almost in all the documents though they normally do not have much semantic content that can be searched or indexed. Under mapper implementation, tokens that are found to be in the stopwords.txt are filtered out prior to the production of intermediate outputs. In English text, the proportion of stop words may be as large as 40-60By eliminating them prior to generating mapper outputs: The intermediate key-value pairs are considerably minimized. The size of the output files of the mappers stored locally on each node is reduced. The Shuffle step of MapReduce is used to move intermediate key-value pairs between mappers and reducers in the network. During shuffle: Mapper outputs are sorted. The pairs are key-values that get sent to reducers that are assigned to each key. If stop words are not removed: Very common words as keys such as the or and would cause very large numbers of records. Such records have to be moved around the network to the reducer resulting in excessive network I/O.

2.3 Quantification

```
foundation 20.txt
help 20.txt
produce 20.txt
new 20.txt
ebooks 20.txt
subscribe 20.txt
email 20.txt
newsletter 20.txt
hear 20.txt
new 20.txt
ebooks 20.txt
Stopword ratio: 0.5007
```

Figure 1: Stopwords ratio

As mentioned in the figure above, it was about 50

2.4 Processing Logic

2.4.1 Mapper

The mapper loads the stop words initially by calling the `loadstopwords` function which reads all the words found in `stopwords.txt` and puts them into the set to allow quick access. Then it identifies the document that is currently being read based on the Hadoop environment variable `mapreduce.map.inputfile`.

One line at a time the input book is inputted to the `normalize` function which changes the text to lowercase and eliminates undesired characters by regular expression. The line so purged is divided into words (tokens).

Each token is checked by the mapper as being a stop word or not. Otherwise the key-value pair generated by the mapper will be:

```
word : document_name(s)
```

2.4.2 Reducer

The reducer then bundles these outputs to construct the final reverse index of each word to the documents in which they occur.

3 Scalability Analysis and Results

3.1 Methodology

The data set is 20 plain text Project Gutenberg books uploaded to HDFS in the root directory `/user/student/library/`. The size of the dataset is quite small, and even a book has up to one HDFS block.

Hadoop cluster was Dockerized based on Hadoop 3.2.1 and it was set up with one NameNode and a maximum of three DataNodes. Three cluster configurations were experimented with:

Experiment	Number of DataNodes	HDFS Replication Factor
1	1	1
2	2	2
3	3	3

Table 1: Cluster configurations used in experiments.

For each experiment, the reverse index MapReduce job was submitted using `run_job.sh`. Execution times were recorded at the end of each job to calculate *speedup*, defined as the ratio of the single-node runtime to the multi-node runtime. Intermediate outputs and the final reverse index files were stored in HDFS, and relevant metrics (such as output size and shuffle reduction) were observed using HDFS commands.

3.2 Performance Visualization

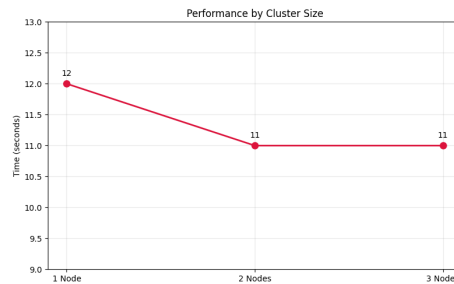


Figure 2: Cluster Size VS Execution time

3.3 Results Discussion

The values of speedup obtained were 1.00 (1 node), 1.09 (2 nodes) and 1.09 (3 nodes). The point to point performance ratio was minimal with the increase in DataNodes and the time spent on executing the network dropped to 11 seconds (2 and 3 node). However, this optimization is not an expression of linear speedup.

The initial causes of the low performance improvement are the Hadoop overheads, such as task initiation, network overheads during the shuffle phase, and the coordination overheads. Also, as Amdahl Law states, the existence of a sequential component of the job limits the overall scalability. The comparatively small data also restricts the use of parallelism and thus the underutilization of cluster resources.

The bottlenecks that were identified in the course of the experiment are job startup overhead, input splits, network communication costs, and shared hardware resources in the cluster.

4 Conclusion

This project applied a distributed inverted index on the Hadoop MapReduce and showed the usefulness of stop-word removal to minimize intermediate data and enhance the efficiency of shuffle.

Scalability Scalability showed minor gains and non-linear speed to scale as the number of DataNodes was increased (12s to 10.5s). This can be largely attributed to Hadoop overheads, lack of parallelism and small dataset as Amdahl Law describes.

In general, the use of distributed indexing is efficient, and the effect of its performance enhances with the larger datasets.